



Validation of Formal Models by Interactive Simulation

Fabian Vu^(✉)  and Michael Leuschel^(✉) 

Institut für Informatik, Universität Düsseldorf, Universitätsstr. 1, 40225 Düsseldorf,
Germany

{fabian.vu,leuschel}@uni-duesseldorf.de

Abstract. Validating requirements for safety-critical systems with user interactions often involves techniques like animation, trace replay, and LTL model checking. However, animation and trace replay can be challenging since user and system events are not distinguished, and formulating LTL properties requires expertise.

This work introduces interactive simulation, a new technique that combines domain-specific visualization of formal models with timed probabilistic simulation to create more realistic prototypes. It allows domain experts and users to interact with formal models and simulate the system/environment reactions. State diagrams are also generated for inspecting user interactions and system reactions. Finally, we demonstrate *interactive simulation* on the ABZ automotive case study.

Keywords: Validation · Formal Methods · Visualization · Simulation · Interactive

1 Introduction and Motivation

Many safety-critical systems require human interaction to trigger a response from the system or environment. For instance, a lift moves on button clicks, car lighting is controlled by a driver [1], the airplane landing gear is operated by a pilot [2], and air traffic controllers schedule airplanes via computers [3].

Safety-critical systems are often modeled using formal methods which make use of mathematical notation. For example, models in B [4] and Event-B [5] rely on set theory and first-order logic. This makes it hard for users and domain experts to understand and interact with the model. These interactions cannot always be fully formalized or verified; hence validation is important to ensure that a formal model meets desired user requirements [6].

Approaches for domain-specific views for formal models include VisB [7] for interactive visualizations, and SimB [8] for simulating real-world behavior with probabilistic and timing properties. Both visualization and simulation are fundamental constructs for *validation obligations* (VOs) [9], an approach to validate

The research presented in this paper has been conducted within the IVOIRE project, funded by “Deutsche Forschungsgemeinschaft” (DFG) and the Austrian Science Fund (FWF) grant # I 4744-N.

requirements in formal models systematically. VOs also take domain experts' and users' feedback into account. Before this work, SIMB was not responsive to user interaction in VisB, making it impossible to trigger system reactions with timing behavior based on user interaction.

This paper introduces a new *interactive simulation* technique, integrated into SIMB in PROB2-UI. *Interactive simulation* allows users to execute events via VisB, triggering automatic system reactions via SIMB simulation. State diagrams focusing on graphical components in VisB are also presented to provide a domain-specific view of user interactions with the system. The features improve user experience, specifically in formal models with human-machine interactions, providing better access to the validation process for users and domain experts.

2 Interactive Simulation

Interactive simulation combines animation, simulation, and visualization. First, we present the principles of VisB and SIMB, and then the implementation of *interactive simulation*.

Principles. VisB is a visualization tool in PROB2-UI [10] which uses the animator, model checker and constraint solver PROB [11]. A VisB visualization consists of an SVG file, and a glue file that links SVG objects with the formal model. The glue file includes observers for SVG objects (VisB items) that change the objects' attributes (like colour) based on the model's current state, and click listeners on SVG objects (VisB events) that execute events in the formal model.

```
{ "id": "peds_red", "attr": "fill",
  "value": "IF tl_peds = red THEN \"red\" ELSE \"black\" END"},
{ "id": "peds_green", "attr": "fill",
  "value": "IF tl_peds = green THEN \"green\" ELSE \"black\" END" }
```

Listing 1. Example of VisB Items

```
{ "id": "PitmanUpward",
  "event": "ENV_Pitman_DirectionBlinking", "predicates": ["newPos=Upward7"] }
```

Listing 2. Example of VisB Event

Listing 1 shows VisB items for the pedestrians' traffic light's appearance based on the variable `tl_peds` (e.g. fill attribute of `peds_red` is "red" when `tl_peds` is equal to `red`, otherwise "black"). Listing 2 shows an example of a VisB event from an automotive case study (see Sect. 4). The VisB event states that a click on the SVG object with `PitmanUpward` as `id` executes the event `ENV_Pitman_DirectionBlinking` with `newPos=Upward7` in the formal model.

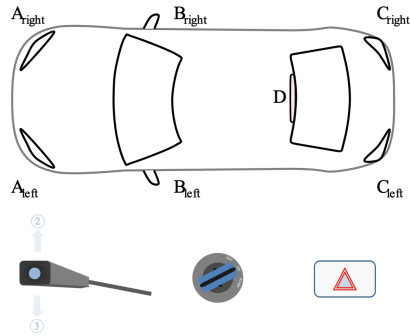


Fig. 1. VisB Visualization for Automotive Case Study [12]

Figure 1 shows a complete visualization of the automotive case study with the car lighting system, the pitman controller (to turn on the direction indicators), the key ignition (to turn on the engine), and the warning lights button.

However, VISB has some limitations, e.g., VISB does not enable the activation of a sequence of events or control the time elapsed between events, nor allow probabilistic event selection. These features are provided by another component.

SIMB is a tool in PROB2-UI which uses PROB's animator to simulate realistic scenarios. A modeler can use SIMB to encode simulations with activation diagrams (see Fig. 2) annotating events in formal models with time and probabilities. Simulations start automatically at the model's initialization, triggering other events. Ideally, simulations run deadlock-free, i.e., events continue triggering each other. The core concept is *activations* of two kinds [8]: (1) *Direct activations* which execute events after a specific time, and optionally trigger other activations, and (2) *probabilistic choices* which choose between activations probabilistically (eventually a *direct activation* must be reached). SIMB manages a scheduling table to represent the simulation's current state as a multiset of scheduled *direct activations*, along with the scheduled time, i.e., the time until the corresponding event is executed. For illustration, we only show *direct activations* (yellow diamonds in Fig. 2) in this paper.

While a simulation is running, the user can still intervene and execute events in PROB2-UI. However, SIMB was not responsive to user interaction as there was no link between user interaction and SIMB's activation diagram. Thus, it was not possible to apply a user interaction to trigger a chain of system events. To address this issue, we developed an *interactive simulation* technique.

Figure 2 shows parts of a SIMB activation diagram for [12] where both activations (yellow diamonds `blinking_on` and `blinking_off`; JSON representation in Fig. 3) trigger each other in a cycle. Each activation executes events from the model (`RTIME_Blinker_On` and `RTIME_Blinker_Off` after a delay of t). The complete activation diagram controls both user behavior and the vehicle's reaction automatically, with no distinction between user and system events or activations.

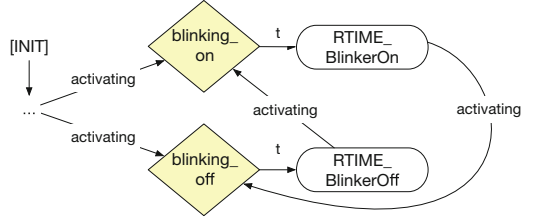


Fig. 2. Example of SimB Diagram

```
{
  "id": "blinking_on",
  "execute": "RTIME_BlinkerOn",
  "after":
    "curDeadlines(blink_deadline)",
  "activating": "blinking_off", ...}

```

```
{
  "id": "blinking_off",
  "execute": "RTIME_BlinkerOff",
  "after":
    "curDeadlines(blink_deadline)",
  "activating": "blinking_on", ...}

```

Fig. 3. Example of SimB Activations in Fig. 2

Architecture. Figure 4 shows the architecture of PROB2-UI and PROB together with VISB and SIMB. When loading a VISB visualization or a SIMB simulation, they are first checked syntactically and semantically wrt. the model. A user can then interact with the formal model via PROB’s animator, or the VISB visualization. With *interaction simulation*, users can execute an event that automatically triggers a sequence of other events with time elapsing in between. This is realized by (newly introduced) SIMB listeners that recognize user interactions and trigger SIMB activations accordingly. A user can then observe the system’s reaction.

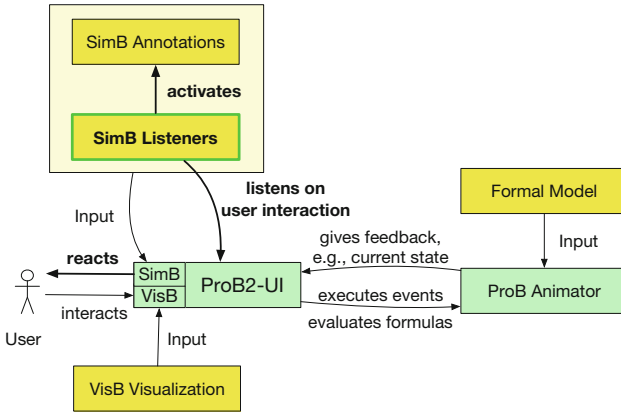


Fig. 4. Architecture with PROB2-UI, PROB, VISB, SIMB, and User Interaction (new features marked in **bold**)

Implementation. In the implementation, we distinguish events of two types: those triggered by SIMB, and those triggered via user interaction. Events triggered by SIMB are already part of the activation diagram.

SIMB listeners are defined on events that are manually triggered, fulfilling a predicate (realized with **event** and **predicate** in JSON). Based on the user interaction, a SIMB listener triggers simulations associated with the **activating** field which stores activations. Thus, SIMB listeners define additional entry points into the activation diagram which are triggered by user interaction. Listing 3 shows a SIMB listener which detects user interactions on `ENV_Pitman_DirectionBlinking`, and triggers the activations **blinking_on** and **blinking_off** (see Fig. 3).

This results in the activation diagram in Fig. 5. Unlike Fig. 2, user interaction is integrated into SIMB as an entry point for the simulation. The blinking lights are triggered by user interaction, and not as part of a fully automatic simulation activated at the model's initialization.

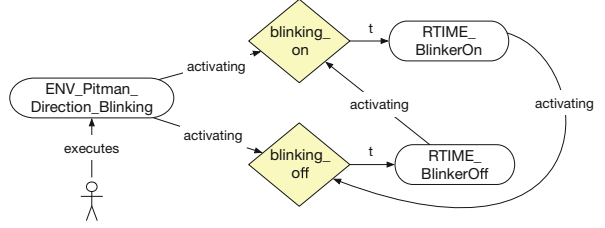


Fig. 5. Activation Diagram with SimB Listener

```
{
  "id": "start_blinking",
  "event": "ENV_Pitman_DirectionBlinking",
  "predicate": "1=1",
  "activating": ["blinking_on", "blinking_off"]}

```

Listing 3. Example for SimB Listener

3 VisB Diagrams

PROB has a feature that projects the state space onto an expression [13]. Such an expression could be a tuple of variables of interest. These diagrams are useful to study the model's behavior for a particular aspect or feature. This work extends that feature by combining it with VISB. This results in VISB diagrams (e.g., Fig. 6) that can be read by domain experts, without having to understand the textual representation of B values.

VISB diagrams combined with *interactive simulation* help to see how user events and system/environment interact with each other from the user's perspective in VISB. A detailed case study is presented in Sect. 4 (notably Fig. 8). VISB diagrams focus on a subset of graphical objects and attributes. We use PROB to compute the state space projection for relevant expressions used by VISB to compute the attributes. We also use VISB to render each projected state graphically. Figure 6 shows two projected states (out of five in the complete state space), along with their graphical renderings¹.

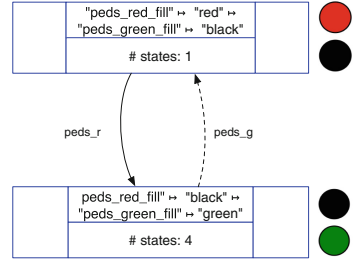


Fig. 6. VISB Diagram from Listing 1

Let us describe this feature more formally. Let V_{items} be the set of VISB items and let V_{prj} with $V_{prj} \subseteq V_{items}$ be the subset of VISB of interest. A VISB item $v \in V_{items}$ contains attributes for the SVG object's id, attribute and value,

¹ The technique is not yet fully automated: VISB visualisations were added manually to the right-hand side of Fig. 6. Note that our feature was inspired by *transition diagrams* in BMotionWeb [14, 15].

i.e., $v = (v.id, v.attr, v.value)$. A VisB diagram is created with a projection [13] on:

$$v_1.id \mapsto v_1.attr \mapsto v_1.value \mapsto \dots \mapsto v_n.id \mapsto v_n.attr \mapsto v_n.value$$

where $V_{prj} = \{v_1, \dots, v_n\}$ and $\forall i, j \in 1..n \wedge i \neq j \implies v_i \neq v_j$.

An example is given for Listing 1, resulting in the left-hand side of Fig. 6:

```
"peds_red"→"fill" → "IF tl_peds = red THEN \"red\" ELSE \"black\" END" →
"peds_green"→"fill" → "IF tl_peds = green THEN \"green\" ELSE \"black\" END"
```

4 Case Study

This section demonstrates the features introduced in Sect. 2 and Sect. 3 on an automotive case study [12]. A VisB visualization is shown in Fig. 1.

Now, we focus on specific requirements that have been modeled and validated by Leuschel et al. [12] and Vu et al. [8], with a special interest in the interactive/human (*italic*) and automatic/autonomous (underlined) parts, and their connection:

- **ELS-1** *Direction blinking left: Assuming that the ignition key is inserted: When moving the pitman arm in position “turn left”, the vehicle flashes all left direction indicators (...) synchronously [...] and a frequency of 1.0 Hz $\pm$ 0.1 Hz (i.e. 60 flashes per minute $\pm$ 6 flashes).*
- **ELS-8**: As long as *the hazard warning light switch is pressed (active)*, all direction indicators flash synchronously. [...]
- **ELS-12**: When *hazard warning is deactivated again*, the pitman arm is in position “direction blinking left” or “direction blinking right” *ignition is On*, the direction blinking cycle should be started (see Req. **ELS-1**).

Validation by Interactive Simulation. Based on requirements and model [12], we encode SIMB listeners and activations. We use VisB to perform user interactions described in **ELS-1**, **ELS-8**, and **ELS-12** and check if the car reacts as desired. Initially, the engine is off, warning lights are not active, and the pitman arm is in **Neutral** position (see Fig. 7a). First, the driver turns on the engine (see Fig. 7b) and moves the pitman arm to **Downward7** (see Fig. 7c) corresponding to the user interaction of **ELS-1**. The car’s left direction indicators are expected to blink every 500 ms, which is confirmed in Fig. 7d and Fig. 7e. Secondly, the driver activates the warning lights, and checks if all direction indicators blink every 500 ms (described in **ELS-8**). This user interaction is shown in Fig. 7f, and the car’s reaction is confirmed in Fig. 7f and Fig. 7g. Finally, the driver deactivates the warning lights (see Fig. 7h), and checks if all left direction indicators blink every 500 ms (as pitman arm is still in **Downward7**; requirement **ELS-12**). The desired reaction is confirmed by the user in Fig. 7h and Fig. 7i.

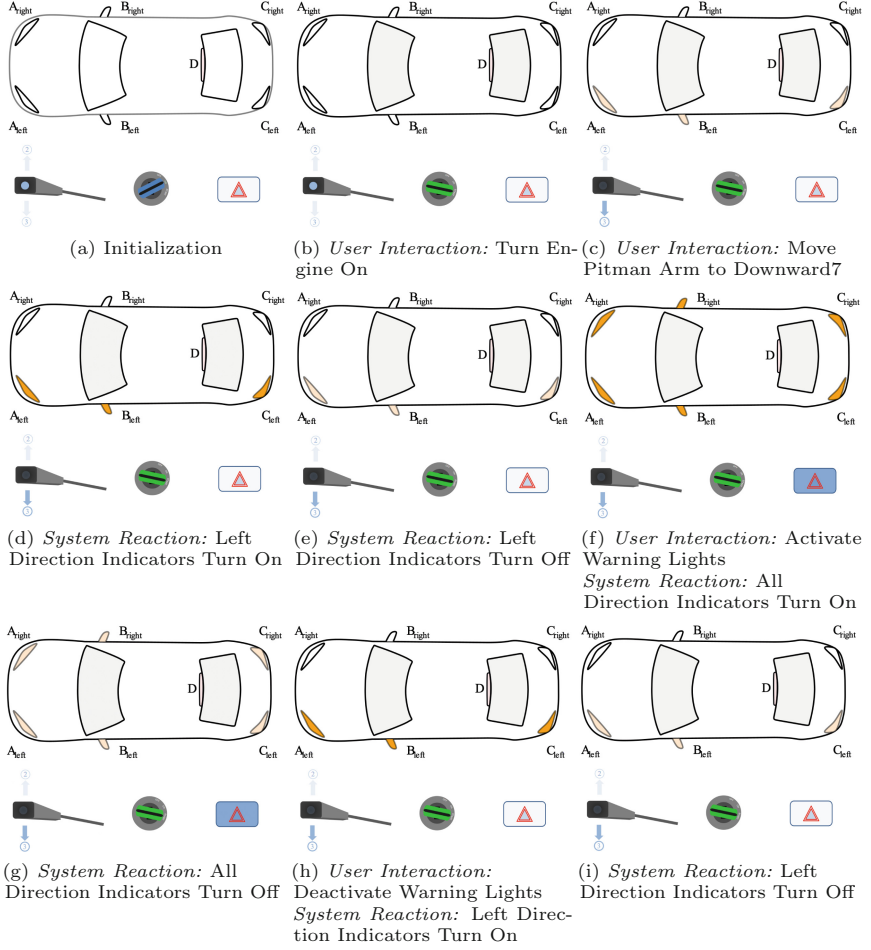


Fig. 7. Validation of **ELS-1**, **ELS-8**, **ELS-12** from User's Perspective in PROB2-UI (Visualization and User Interaction in VisB, System Reaction via SIMB)

Validation by VisB State Diagram. After running user scenarios for **ELS-1**, **ELS-8**, and **ELS-12** via interactive simulation (described in Fig. 7), we inspect the VisB state diagram (see Fig. 8). For clarity, we replaced the state diagram nodes (textual representation) with the corresponding graphical objects. This is currently done manually, but we attempt to automate it in the future.

This results in Fig. 8 with six states. The edges represent events executed in Fig. 7. Thus, Fig. 8 does not show events that are not part of the scenario in Fig. 7. The diagram shows that turning on the engine does not result in any reaction from the car, while user events on the pitman arm and warning lights button trigger the flashing cycles. Deactivating the hazard lights switches to the left blinking lights cycle as the pitman arm is still in **Downward7**.

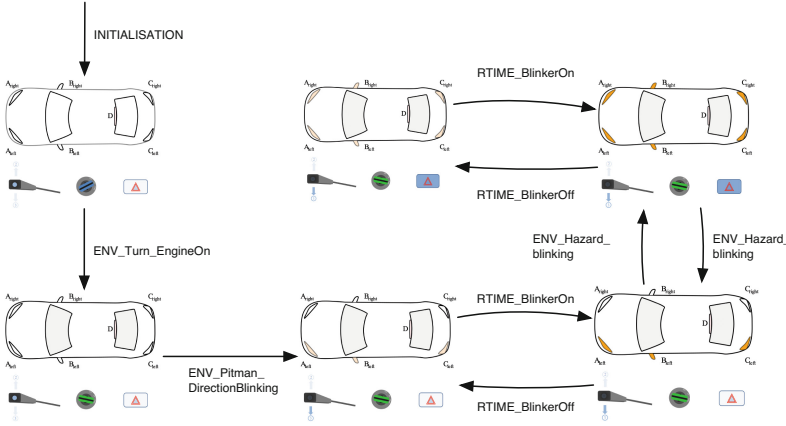


Fig. 8. State Diagram from Fig. 7

5 Related Work

Animation, Testing. In animation, the user has to execute all events manually. *Interactive simulation* only requires users to execute user events manually after which system events are executed automatically. This improves usability for users but requires additional effort in encoding the simulation. Existing animators are, e.g., the PROB animator [11], and ASMETA for ASMs [16]. Domain-specific scenarios are supported for Event-B with Gherkin using PROB [17,18], and for ASMs with ASMETAV [19] and the AVALLA language, and ASMETA2C++ [20]. As we ask: “*when the user executes an event, then how does the system react?*”, there is some overlap between such scenarios and SIMB activation diagrams.

The scenario checker uses PROB for animation and BMotionStudio [21] for visualization of formal models [22]. It distinguishes between external (executed manually) and internal events (fired automatically), similar to our work. SIMB simulates events more precisely as it encodes probabilistic and timing behavior.

Simulators. There are various simulators like SIMB: JeB [23], AsmetaS [24], Uppaal [25], or the co-simulation tool INTO-CPS [26]. In particular, Uppaal and INTO-CPS can handle continuous time, while our approach works with discrete time only. A more detailed comparison is given by Vu et al. [8].

Visualizations. VISB has been compared with BMotionWeb [14,15], BMotionStudio [21], and PROB’s animation function [27] in [7]. Those tools all make it possible to interact with a formal model via a visualization. Unlike this work, they do not support easy simulation of autonomous events as a reaction to a user event. BMotionWeb also includes a feature to generate a projection diagram on graphical objects which is an inspiration for VISB state diagrams.

BRAMA [28] allows animation of formal B models through Flash visualizations, and contains listeners to simulate system events. Brama was also used in

an architecture by Méry and Singh where real-time data were collected, trained, and used to animate formal models [29]. SIMB also uses listeners to trigger simulations with timing and probabilistic behavior. While Brama was a standalone Flash application, SIMB is fully integrated into PROB2-UI, allowing for use with other features in PROB2-UI. Using real-time data in SIMB is still future work.

PVSio-Web [30] is a tool to create prototypes for PVS models. Like SIMB, it also extends simulation features to support human-machine interfaces.

Looking a bit further, there is also a considerable amount of research on formal methods and human-computer interaction (e.g., [31]); some may benefit from our new tooling. Other work on combining verification with simulation (e.g., [32]) can inspire further linking our simulation techniques with B verification techniques. We may also investigate using CSP (already supported by PROB) and its associated refinement notions with support for external and internal choice, as a means of formally verifying our user interactions.

6 Conclusion and Future Work

This work presented SIMB’s *interactive simulation* which is coordinated with domain-specific interactive VISB visualizations. The feature is realized by SIMB listeners which recognize user interactions (e.g. in VISB) and trigger SIMB simulations, i.e., autonomous events with probabilistic and timing behavior. *Interactive simulation* helps (1) to improve the user experience of formal models, and (2) to validate requirements related to user interactions and expected system reactions. For domain-specific users, *interactive simulation* is more accessible than LTL as writing LTL requires expertise. Compared to classic animation, *interactive simulation* reduces the user’s effort to interact with formal models as the user only has to execute user events while automatic events are simulated. In exchange, *interactive simulation* requires additional effort to be invested in modeling the simulations including human/machine interaction. We also presented state diagrams for domain-specific visualizations in VISB, supporting domain-specific inspection. In an automotive case study, we demonstrated the effectiveness of *interactive simulation* and those state diagrams. Here, we successfully validate requirements by executing user events and observing desired system reactions.

- Case studies are available at: https://github.com/favu100/SimB-examples/tree/main/Interactive_Examples
- PROB2-UI (with presented features) is available at: <https://prob.hhu.de/w/index.php/Prob2-UI>
- More information on SIMB including *interactive simulation* are available at: <https://prob.hhu.de/w/index.php?title=SimB>

In the future, we plan to formalize SIMB’s semantics. This could help verify SIMB’s *interactive simulator*. Another future work is the refinement of SIMB simulation (as mentioned in [8]) which also affects SIMB listeners.

Acknowledgements. We would like to thank Sebastian Stock and anonymous reviewers for proofreading and giving feedback.

References

1. Houdek, F., Raschke, A.: Adaptive Exterior Light and Speed Control System (2019). <https://abz2020.uni-ulm.de/case-study>
2. Boniol, F., Wiels, V.: The landing gear system case study. In: Boniol, F., Wiels, V., Ait Ameer, Y., Schewe, K.-D. (eds.) ABZ 2014. CCIS, vol. 433, pp. 1–18. Springer, Cham (2014). https://doi.org/10.1007/978-3-319-07512-9_1
3. Palanque, P., Campos, J.: AMAN Case Study (2022). <https://drive.google.com/file/d/1IqftxQIvrWpX1lcRts3WJzrBH7a3dMln/>
4. Abrial, J.-R.: The B-Book: Assigning Programs to Meanings. Cambridge University Press, Cambridge (2005)
5. Abrial, J.-R., Butler, M., Hallerstede, S., Hoang, T.S., Mehta, F., Voisin, V.: Rodin: an open toolset for modelling and reasoning in Event-B. *Int. J. Softw. Tools Technol. Transfer* **12**(6), 447–466 (2010)
6. Institute of Electrical and Electronics Engineers. IEEE Standard Computer Dictionary: A Compilation of IEEE Standard Computer Glossaries (1991)
7. Werth, M., Leuschel, M.: VisB: a lightweight tool to visualize formal models with SVG graphics. In: Raschke, A., Méry, D., Houdek, F. (eds.) ABZ 2020. LNCS, vol. 12071, pp. 260–265. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-48077-6_21
8. Vu, F., Leuschel, M., Mashkoor, A.: Validation of formal models by timed probabilistic simulation. In: Raschke, A., Méry, D. (eds.) ABZ 2021. LNCS, vol. 12709, pp. 81–96. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-77543-8_6
9. Mashkoor, A., Leuschel, M., Egyed, A.: Validation obligations: a novel approach to check compliance between requirements and their formal specification. In: ICSE 2021 NIER, pp. 1–5 (2021)
10. Bendisposto, J., et al.: PROB2-UI: a Java-based user interface for ProB. In: Lluch Lafuente, A., Mavridou, A. (eds.) FMICS 2021. LNCS, vol. 12863, pp. 193–201. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-85248-1_12
11. Leuschel, M., Butler, M.: ProB: a model checker for B. In: Araki, K., Gnesi, S., Mandrioli, D. (eds.) FME 2003. LNCS, vol. 2805, pp. 855–874. Springer, Heidelberg (2003). https://doi.org/10.1007/978-3-540-45236-2_46
12. Leuschel, M., Mutz, M., Werth, M.: Modelling and validating an automotive system in classical B and Event-B. In: Raschke, A., Méry, D., Houdek, F. (eds.) ABZ 2020. LNCS, vol. 12071, pp. 335–350. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-48077-6_27
13. Ladenberger, L., Leuschel, M.: Mastering the visualization of larger state spaces with projection diagrams. In: Butler, M., Conchon, S., Zaïdi, F. (eds.) ICFEM 2015. LNCS, vol. 9407, pp. 153–169. Springer, Cham (2015). https://doi.org/10.1007/978-3-319-25423-4_10
14. Ladenberger, L., Leuschel, M.: BMotionWeb: a tool for rapid creation of formal prototypes. In: De Nicola, R., Kühn, E. (eds.) SEFM 2016. LNCS, vol. 9763, pp. 403–417. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-41591-8_27
15. Ladenberger, L.: Rapid creation of interactive formal prototypes for validating safety-critical systems. Ph.D. thesis, Universitäts- und Landesbibliothek der Heinrich-Heine-Universität Düsseldorf (2016)

16. Bonfanti, S., Gargantini, A., Mashkoor, A.: AsmetaA: animator for abstract state machines. In: Butler, M., Raschke, A., Hoang, T.S., Reichl, K. (eds.) ABZ 2018. LNCS, vol. 10817, pp. 369–373. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-91271-4_25
17. Snook, C., Hoang, T.S., Dghaym, D., Fathabadi, A.S., Butler, M.: Domain-specific scenarios for refinement-based methods. *J. Syst. Archit.* **112**, 101833 (2021)
18. Fischer, T., Dghyam, D.: Formal model validation through acceptance tests. In: Collart-Dutilleul, S., Lecomte, T., Romanovsky, A. (eds.) RSSRail 2019. LNCS, vol. 11495, pp. 159–169. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-18744-6_10
19. Carioni, A., Gargantini, A., Riccobene, E., Scandurra, P.: A scenario-based validation language for ASMs. In: Börger, E., Butler, M., Bowen, J.P., Boca, P. (eds.) ABZ 2008. LNCS, vol. 5238, pp. 71–84. Springer, Heidelberg (2008). https://doi.org/10.1007/978-3-540-87603-8_7
20. Bonfanti, S., Gargantini, A., Mashkoor, A.: Design and validation of a C++ code generator from Abstract State Machines specifications. *J. Softw. Evol. Process* **32** (2020)
21. Ladenberger, L., Bendisposto, J., Leuschel, M.: Visualising Event-B models with B-Motion studio. In: Alpuente, M., Cook, B., Joubert, C. (eds.) FMICS 2009. LNCS, vol. 5825, pp. 202–204. Springer, Heidelberg (2009). https://doi.org/10.1007/978-3-642-04570-7_17
22. Snook, C., Hoang, T.S., Fathabadi, A.S., Dghaym, D., Butler, M.: Scenario checker: an Event-B tool for validating abstract models. In: Proceedings of the 9th Rodin User and Developer Workshop, pp. 12–14 (2021)
23. Mashkoor, A., Yang, F., Jacquot, J.-P.: Refinement-based validation of Event-B specifications. *Softw. Syst. Model.* **16**(3), 789–808 (2017)
24. Gargantini, A., Riccobene, E., Scandurra, P.: A metamodel-based language and a simulation engine for abstract state machines. *J. Univ. Comput. Sci.* **14**, 1949–1983 (2008)
25. Bengtsson, J., Larsen, K., Larsson, F., Pettersson, P., Yi, W.: UPPAAL—a tool suite for automatic verification of real-time systems. In: Alur, R., Henzinger, T.A., Sontag, E.D. (eds.) HS 1995. LNCS, vol. 1066, pp. 232–243. Springer, Heidelberg (1996). <https://doi.org/10.1007/BFb0020949>
26. Thule, C., Lausdahl, K., Gomes, C., Meisl, G., Larsen, P.G.: Maestro: the INTO-CPS co-simulation framework. *Simul. Model. Pract. Theory* **92**, 45–61 (2019)
27. Leuschel, M., Samia, M., Bendisposto, J., Luo, L.: Easy graphical animation and formula visualisation for teaching B. *The B Method: from Research to Teaching*, pp. 17–32 (2008)
28. Servat, T.: BRAMA: a new graphic animation tool for B models. In: Julliand, J., Kouchnarenko, O. (eds.) B 2007. LNCS, vol. 4355, pp. 274–276. Springer, Heidelberg (2006). https://doi.org/10.1007/11955757_28
29. Méry, D., Singh, N.K.: Real-time animation for formal specification. In: Aiguier, M., Bretaudeau, F., Krob, D. (eds.) Complex Systems Design & Management, pp. 49–60. Springer, Heidelberg (2010). https://doi.org/10.1007/978-3-642-15654-0_3
30. Watson, N., Reeves, S., Masci, P.: Integrating user design and formal models within PVsio-web. In: Proceedings F-IDE, EPTCS, vol. 284, pp. 95–104 (2018)
31. Dix, A.J.: Formal methods. In: Perspectives on HCI: Diverse Approaches, pp. 9–43. Academic Press, London (1995)
32. Schwammberger, M., Harper, C., Alves, G.V., Chance, G., Pipe, T., Eder, K.: Integrating Formal Verification and Simulation-Based Assertion Checking in a Corroborative V&V Process. *CoRR*, abs/2208.05273 (2022)